

CURSO COMPLETO

Da ideia ao ar

Um guia completo, sem enrolação, de como construímos uma API real e a colocamos rodando na internet — escrito para você aprender de verdade.

Escrito para o João, que quer aprender a programar e a pensar como programador.

O que você vai aprender neste guia

1. Introdução: como um programador pensa

- Programação é resolver problemas, não decorar sintaxe
- Erros são informação, não fracasso
- Toda decisão tem trade-off

2. Parte I — Fundamentos que você precisa dominar

- Terminal, editor, Git
- HTTP: o idioma da web
- Cliente e servidor

3. Parte II — O que é uma API e por que a gente fez uma

- API REST
- JSON
- Verbos, status codes, endpoints

4. Parte III — Construindo a API com .NET 10

- C# em 5 minutos
- Minimal APIs
- Injeção de dependência

5. Parte IV — Onde os dados moram: banco de dados

- Banco relacional e SQL
- PostgreSQL
- ORM: falando com o banco em C#
- Migrations: versionando o banco

6. Parte V — Autenticação e autorização

- Password hashing (nunca guarde senha em texto puro!)
- JWT: o crachá do usuário
- Roles: quem pode o quê

7. Parte VI — Integrando com serviços externos

- Chamando outras APIs (Efí Bank)
- mTLS: certificados de segurança
- Webhooks e idempotência

8. Parte VII — Empacotando com Docker

- O problema que Docker resolve
- Dockerfile
- Docker Compose

9. Parte VIII — Servidor real na internet

- O que é uma VPS
- SSH, chave pública e privada
- Firewall e fail2ban

10. Parte IX — CI/CD: automação total

- GitHub Actions
- GHCR (Container Registry)
- Watchtower (auto-update)

11. Parte X — Testes: dormir tranquilo

- Unit test
- Integration test
- Smoke test

12. Parte XI — Documentação viva: OpenAPI e Scalar

13. Parte XII — Boas práticas (o que ainda vamos fazer)

- HTTPS, backup, rate limit, refresh token
- Logging estruturado, observabilidade

14. Parte XIII — Como pensar como um dev sênior

15. Glossário

16. Recursos: livros, canais, cursos

Introdução

Como um programador pensa (spoiler: não é como você imagina)

Antes de a gente colocar a mão no código, precisa desmontar três mitos que atrapalham quem tá começando:

Mito 1: “Preciso decorar tudo”

Programadores **não decoram sintaxe**. Ninguém lembra a assinatura exata de uma função. A gente lembra que *existe*, e vai no Google/documentação/IA lembrar como se escreve. O que a gente cultiva é o **vocabulário de conceitos**: sei que existe "arredondamento bancário", sei que existe "cache", sei que existe "idempotência". Aí eu pesquiso como usar em cada linguagem.

O PONTO

Aprender programação é aprender **quais problemas existem** e **quais famílias de solução**. A sintaxe é só a linguagem que você usa pra escrever a solução. Muda a linguagem, mas os problemas permanecem.

Mito 2: “Se deu erro, eu falhei”

Erros são a **coisa mais valiosa que o computador pode te dar**. Um erro te diz exatamente o que está errado. Quando o CI falhou nos testes de integração, o log dizia:

```
System.InvalidOperationException : ConnectionStrings:Default ausente.
```

Essa linha me disse duas coisas: (1) o problema é a configuração da connection string, (2) ela está "ausente" — ou seja, o código está procurando um valor que não foi definido. Em 30 segundos eu sabia o caminho da correção.

REGRA DE OURO

Leia a mensagem de erro inteira antes de perguntar pra alguém. Copia e cola no Google — 90% das vezes alguém já teve o mesmo problema. Isso vale mais que qualquer curso.

Mito 3: “Existe o jeito certo”

Não existe. Existem **trade-offs**. Toda decisão em programação é uma escolha entre coisas boas e coisas ruins. Alguns exemplos do nosso projeto:

Escolha que fizemos	Vantagem	Desvantagem
Minimal APIs em vez de Controllers	Menos código	Menos "opinado", cada dev pode estruturar de um jeito
Migrations rodando no startup	Deploy simples	Não escala para muitas réplicas
Postgres no mesmo servidor da API	Barato, simples	Se o servidor cai, cai tudo

Um *dev sênior* não é quem sabe "o certo" — é quem consegue **explicar por que escolheu** uma opção em vez de outra.

REFLITA

Pense num aplicativo que você usa muito (Instagram, WhatsApp, Nubank). Toda escolha ali é uma decisão de alguém com um trade-off. Por que o WhatsApp não deixa você editar mensagens antigas facilmente? É bug? Não. Foi uma escolha de simplicidade vs auditabilidade. Se você começar a olhar assim, vai aprender muito só de usar apps.

Parte I

Fundamentos que você precisa dominar antes de qualquer coisa

1.1 O terminal (a "linha de comando")

É uma janela preta onde você digita comandos e o computador executa. É **o único jeito profissional** de mexer em servidores — não tem mouse lá.

No Windows tem o **PowerShell** (mais moderno) e o **CMD** (antigão). No macOS/Linux tem o **Bash** ou **Zsh**. As sintaxes são diferentes, mas os conceitos são os mesmos: você navega em pastas, roda programas, encadeia comandos.

```
# PowerShell (Windows)
cd C:\Users\joao\projetos      # muda de pasta
ls                             # lista arquivos
git status                    # roda o git

# Bash (Linux/Mac)
cd /home/joao/projetos
ls -la
git status
```

ANALOGIA

Se o Windows Explorer é o carro automático, o terminal é o carro manual: você faz TUDO você mesmo, é mais chato no começo, mas você controla tudo com precisão cirúrgica. Todo dev profissional dirige manual.

1.2 O editor de código

O queridinho da galera é o **VS Code** (grátis, da Microsoft). Para .NET, dá também pra usar o **Rider** (pago, da JetBrains) ou **Visual Studio** (só Windows, mais pesado).

Aprenda a usar o editor com atalhos de teclado. Cada segundo salvo em digitar aparece como "10 minutos por dia" no fim do mês.

1.3 Git: viagem no tempo do seu código

Git é um sistema que grava snapshots do seu código a cada mudança que você marca (chamada de **commit**). Se você quebrar algo, volta para o snapshot anterior. Se você quer experimentar sem quebrar o principal, cria uma "branch" (galho paralelo).

```
# Ver o que mudou
git status
git diff

# Salvar um snapshot
git add .
git commit -m "adiciona login por email"

# Enviar pro GitHub
git push origin main

# Baixar mudanças de outros
git pull
```

O **GitHub** é uma empresa (agora da Microsoft) que hospeda repositórios Git na nuvem. É onde o mundo inteiro compartilha código. No nosso projeto, o repositório é github.com/Joaooof/formaturas-api.

1.4 HTTP: o idioma da web

Todo aplicativo que se conecta à internet fala **HTTP**. É um protocolo (um conjunto de regras) para **um cliente** (seu navegador, seu app) pedir alguma coisa para **um servidor** (computador remoto), e o servidor responder.

ANALOGIA

É como pedir comida no drive-thru. Você (cliente) fala pelo alto-falante: "quero um sanduíche" (requisição). O funcionário (servidor) volta com o sanduíche (resposta) ou avisa "acabou" (status de erro).

Toda requisição HTTP tem:

- **Verbo:** a intenção. `GET` (pedir), `POST` (criar), `PUT` (atualizar tudo), `PATCH` (atualizar parte), `DELETE` (remover).
- **URL:** o "endereço" do recurso. Ex: `http://191.101.78.252/api/v1/alunos`
- **Headers:** metadados. Ex: `Authorization: Bearer <token>` (crachá).
- **Body** (às vezes): dados que você envia. Normalmente em JSON.

Toda resposta HTTP tem:

- **Status code:** número de 3 dígitos que diz como foi.
 - **2xx** deu certo (200 OK, 201 Created, 204 No Content).
 - **4xx** você errou (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).
 - **5xx** o servidor errou (500 Internal Server Error).
 - **Body:** os dados que você pediu, normalmente em JSON.
-

Parte II

O que é uma API e por que a gente fez uma

2.1 API resumida em uma frase

API é um **garçom**. Você (o cliente — pode ser um app mobile, um site, outro sistema) pede algo. A API vai na cozinha (banco de dados, outros serviços), busca, monta a resposta, e te entrega.

No nosso caso, a API **FormaturasFlow** serve para um sistema de gestão financeira de formaturas. Quem chama ela vai ser um **front-end** (o site que o pessoal do escritório usa) e um dia talvez um app mobile.

2.2 REST: uma convenção pra APIs sobre HTTP

REST é uma convenção que diz: "URLs representam recursos, verbos HTTP representam ações". Assim:

```
GET    /api/v1/alunos      # lista todos os alunos
GET    /api/v1/alunos/{id} # pega um aluno específico
POST   /api/v1/alunos      # cria um aluno novo
PUT    /api/v1/alunos/{id} # atualiza um aluno
DELETE /api/v1/alunos/{id} # remove um aluno
```

Note que a URL sempre é o mesmo "recurso" (`/alunos`), o que muda é o **verbo**. Isso é REST.

2.3 JSON: como a gente troca dados

JSON (JavaScript Object Notation) é um formato de texto pra trocar dados. Parece um objeto JavaScript ou dicionário Python. Todo mundo entende: navegador, servidor, banco.

```
{
  "id": "abc-123",
  "nomeCompleto": "Maria Silva",
  "email": "maria@ex.com",
  "cpf": "12345678900",
  "criadoEm": "2026-01-15T10:30:00Z"
}
```

REFLITA

Quando você abre o Instagram, o app envia **vários** requests HTTP: um pra buscar o feed, um pra buscar as histórias, um pra baixar cada imagem. Cada um retorna JSON (ou binário no caso das imagens). Toda internet moderna funciona assim.

Parte III

Construindo a API com .NET 10

3.1 O que é .NET, C# e ASP.NET Core

Vamos separar:

- **C#** é a linguagem que a gente escreve (tipo Java ou TypeScript com generics).
- **.NET** é a plataforma que executa C# (tipo a JVM pro Java, ou o Node pro JavaScript).
- **ASP.NET Core** é um framework de dentro do .NET focado em web/API.

É software da Microsoft, mas hoje é 100% open source e roda no Linux (a nossa API tá rodando num Linux Ubuntu na VPS).

3.2 C# em cinco minutos

C# é **tipada** (você declara o tipo de cada variável), **orientada a objetos** (classes, herança) e **compilada** (o código vira código de máquina antes de rodar).

```
// declara variável
int idade = 25;
string nome = "João";
decimal valor = 1250.50m;

// método (função dentro de uma classe)
public decimal Somar(decimal a, decimal b)
{
    return a + b;
}

// classe
public class Aluno
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Nome { get; set; } = string.Empty;
    public string? Email { get; set; }
}
```

Coisas importantes:

- `string?` significa "pode ser null" (nulo, vazio). `string` não pode.
- `{ get; set; }` é uma "propriedade" — como uma variável de uma classe.
- `Guid` é um identificador único global (uma string tipo `a1b2c3d4-e5f6-...`).

3.3 Minimal APIs: menos código, mais foco

Antigamente em .NET a gente escrevia **Controllers** — classes cheias de decoração e ceremony. Desde o .NET 6, tem o estilo **Minimal APIs** que é bem mais direto:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Olá, mundo!");

app.MapPost("/api/v1/alunos", async (Aluno aluno, AppDbContext db) =>
{
    db.Alunos.Add(aluno);
    await db.SaveChangesAsync();
    return Results.Created($"/alunos/{aluno.Id}", aluno);
});

app.Run();
```

Pronto. Isso é uma API funcional. Compara com o antigo (Controller) que exigia 30 linhas pra fazer o mesmo — dá pra sentir a diferença.

3.4 Injeção de dependência (DI)

Perceba que o método POST recebe `Aluno aluno` (o body da requisição) **e também** `AppDbContext db`. Esse `db` não vem do body — vem da **caixa de brinquedos** que o ASP.NET Core mantém.

Você *registra* os serviços que sua app precisa lá no `Program.cs`:

```
builder.Services.AddDbContext<AppDbContext>(o => o.UseNpgsql(connectionString));
builder.Services.AddScoped<JwtTokenService>();
builder.Services.AddHttpClient<EfiClient>();
```

Aí, sempre que algum lugar pedir `AppDbContext db`, o framework *injeta* uma instância. Isso é **Injeção de Dependência**, e serve pra:

- **Trocar implementações** facilmente (em teste, você injeta um banco fake).
- **Não repetir** o código de criar dependências manualmente.
- **Gerenciar ciclo de vida**: `Scoped` vive uma requisição, `Singleton` vive a app inteira, `Transient` é uma nova a cada uso.

ANALOGIA

Imagina uma cozinha profissional. O chef não vai até o mercado comprar cebola cada vez que precisa. O *copeiro* deixa a cebola preparada no lugar certo. Injeção de dependência é o copeiro do seu código.

Parte IV

Onde os dados moram: banco de dados

4.1 Por que um banco de dados?

Você poderia salvar tudo em arquivos `.txt` ou `.json`. Por que não? Porque:

- **Consultas:** quer os alunos da turma X ordenados por nome. Em arquivo, você programa isso na mão. Em banco, você pede em **SQL** e o banco faz.
- **Concorrência:** 10 pessoas tentando salvar ao mesmo tempo → arquivo corrompe. Banco resolve com *transações*.
- **Integridade:** você define regras ("todo aluno tem que ter uma turma") e o banco não deixa quebrar.

4.2 SQL em cinco minutos

SQL é a linguagem pra falar com bancos relacionais. É *declarativa*: você diz *o que quer*, não *como fazer*.

```
SELECT nome, email
FROM alunos
WHERE turma_id = 'abc-123'
ORDER BY nome ASC;
```

Traduzindo: "me devolva o nome e o email dos alunos que estão na turma abc-123, ordenados por nome, ascendente."

4.3 PostgreSQL: nosso banco

Escolhemos o **PostgreSQL** (ou "Postgres"). É open source, gratuito, respeita padrões, tem tudo que precisa (JSON nativo, transações, extensões). É a escolha "boring" que dá certo em 95% dos casos.

Alternativas: **MySQL** (também bom), **SQLite** (arquivo único, ótimo pra prototipagem), **SQL Server** (da Microsoft, forte na Microsoft).

4.4 ORM: falando com o banco em C#

Escrever SQL na mão em C# é chato e propenso a erros. Um **ORM** (Object-Relational Mapper) traduz classes C# em SQL automaticamente. A gente usa o **Entity Framework Core**.

```
// em C#
var alunos = await db.Alunos
    .Where(a => a.TurmaId == turmaId)
    .OrderBy(a => a.NomeCompleto)
    .ToListAsync();

// o EF traduz pra SQL:
// SELECT * FROM alunos WHERE turma_id = @p0 ORDER BY nome_completo ASC
```

Vantagem: **autocomplete**, **type safety** (se você trocar o nome de uma coluna, o código quebra na hora certa).

Desvantagem: em queries muito complexas, o SQL gerado pode ficar bobo. Aí você escreve SQL na mão mesmo.

4.5 Migrations: versionando o banco

Cada mudança na estrutura do banco (nova tabela, nova coluna, novo índice) é uma **migration**. É um arquivo C# que descreve a mudança e como desfazer.

```
dotnet ef migrations add AdicionarTelefoneAluno
```

Isso gera um arquivo em `src/FormaturasFlow.Api/Migrations/`. Quando a aplicação sobe, ela roda `db.Database.MigrateAsync()` que aplica todas as migrations pendentes.

CUIDADO

Você **NUNCA** deve editar uma migration que já foi aplicada em produção. Se precisa mudar, cria uma nova. Editar uma migration antiga é o caminho pra corromper o banco em produção.

Parte V

Autenticação e autorização

5.1 Autenticação vs autorização

Autenticação	Autorização
“Quem é você?”	“O que você pode fazer?”
Login com email/senha	Só admin pode deletar turma

Primeiro autentica (identifica), depois autoriza (verifica permissão).

5.2 Password hashing (não guarde senha em texto puro!)

Se você guarda a senha do usuário como texto no banco, e o banco vazar, todas as senhas ficam expostas. E como as pessoas reusam senhas, isso vaza contas em outros sites também.

A solução: **hash**. Um hash é uma função **de mão única**: fácil de calcular (senha → hash), impossível de reverter (hash → senha). No banco você guarda só o hash. Quando o usuário loga, você hasheia a senha que ele digitou e compara com o hash guardado.

O ASP.NET Core Identity faz isso pra gente com **PBKDF2** — um algoritmo de hash que ainda por cima é *lento* de propósito (dificulta força bruta). Nem precisa pensar: usar `userManager.CreateAsync(user, password)` já cuida disso.

5.3 JWT: o crachá do usuário

Depois que o usuário loga, o servidor devolve um **JWT** (JSON Web Token). É uma string longa dividida em 3 partes por pontos.

```
eyJhbGciOi...  # cabeçalho (qual algoritmo)
.
eyJzdWIiOi...  # payload (claims: id do user, email, roles, expiração)
.
xkY6f4b...     # assinatura (garante que ninguém adulterou)
```

O cliente guarda esse token e envia em cada requisição no header:

```
GET /api/v1/turmas
Authorization: Bearer eyJhbGciOi...
```

O servidor **não precisa consultar o banco** pra saber quem é o usuário — a assinatura HMAC garante que o token não foi adulterado. Isso escala muito bem.

CUIDADO COM JWT

JWT tem uma **data de expiração** mas **não pode ser revogado** antes disso (a menos que você mantenha uma "blocklist" no banco, o que anula a vantagem). Por isso, tokens de acesso devem durar pouco (15-60min) e você usa um **refresh token** pra renovar.

5.4 Roles: quem pode o quê

No nosso projeto temos 3 papéis:

- **super_admin** — pode tudo, inclusive deletar
- **funcionario** — pode criar/editar mas não deletar
- **aluno** — pode ver seus próprios dados

Em cada endpoint, a gente exige um papel:

```
group.MapDelete("/{id:guid}", DeleteAsync)  
    .RequireAuthorization(p => p.RequireRole(Roles.SuperAdmin));
```

Se um `funcionario` tentar chamar esse endpoint, ele recebe **403 Forbidden**. Se não estiver logado, **401 Unauthorized**.

Parte VI

Integrando com serviços externos (a Efi Bank)

6.1 Por que precisamos de um PSP?

PSP (Payment Service Provider) é uma empresa que resolve o pesadelo de receber pagamentos. Sem PSP você teria que integrar com cada banco, cada bandeira, cuidar de compliance PCI, etc. Com PSP você chama uma API e ele faz o resto.

Escolhemos a **Efi Bank** (antiga Gerencianet). Emitir boleto, gerar QR Code de PIX e receber webhook quando o cliente paga.

6.2 mTLS: certificado dos dois lados

Numa conexão HTTPS normal, só o servidor apresenta um certificado (é assim que você tem certeza que está falando com o banco.com e não um site falso). No **mTLS** (mutual TLS), o **cliente** também apresenta um certificado. É como se você e o site trocassem carteiras de identidade.

A Efi exige mTLS. Eles te dão um arquivo `.p12` (o certificado) e você anexa em cada requisição. No nosso código, isso está encapsulado num **HttpMessageHandler**:

```
public class EfiHttpHandler : HttpClientHandler
{
    public EfiHttpHandler(IOptions<EfiOptions> opt)
    {
        var raw = Convert.FromBase64String(opt.Value.CertificateBase64);
        var cert = X509CertificateLoader.LoadPkcs12(raw, opt.Value.CertificatePassword);
        ClientCertificates.Add(cert);
    }
}
```

6.3 Webhooks: quando o servidor "liga" pra gente

Normalmente é você quem chama a Efí ("emita esse boleto"). Mas quando o cliente *paga*, a Efí é que **precisa avisar você**. Ela não pode ficar esperando você perguntar — não escalaria. Então ela usa um **webhook**: quando o pagamento acontece, ela chama uma URL sua (que você cadastrou no painel dela).

```
POST http://api.formaturasflow.com/webhooks/efi?secret=xyz
{ "pix": [{ "txid": "abc123", "valor": "150.00" }] }
```

6.4 Idempotência: fazer 100x = fazer 1x

Webhooks podem ser **reenviados** se a Efí não receber a resposta 200 rápido o suficiente. Se você não protege, o mesmo pagamento pode ser aplicado 3 vezes → aluno pago 3x → dinheiro perdido.

Idempotência = a operação pode ser feita várias vezes e o resultado é o mesmo. Implementamos assim:

```
// pega o ID único do evento vindo da Efí
var eventId = doc.RootElement.GetProperty("evento").GetProperty("id").ToString();

// se já processamos, ignora
var jaExiste = await db.WebhookEvents
    .AnyAsync(w => w.Provider == "efi" && w.EventId == eventId, ct);
if (jaExiste) return Results.Ok(new { duplicado = true });

// senão, processa e guarda o eventId pro futuro
db.WebhookEvents.Add(new WebhookEvent { EventId = eventId, ... });
```

Simple e eficaz. **Toda** integração com PSP deve ter idempotência.

Parte VII

Empacotando com Docker

7.1 O problema que Docker resolve

"Na minha máquina funciona." — todo dev do mundo, uma vez na vida.

Sua máquina tem .NET 10, uma versão específica do Postgres, umas variáveis de ambiente. A máquina de produção tem outra coisa. O programa quebra em produção porque o ambiente é diferente.

Docker resolve isso empacotando seu app **junto com todo o ambiente** num "container". O container roda igual em qualquer lugar.

7.2 Imagem vs Container

- **Imagem** é o "molde", o "arquivo executável". Ex: `postgres:16-alpine`.
- **Container** é uma instância dessa imagem rodando. Você pode rodar 3 containers da mesma imagem.

ANALOGIA

Imagem é a receita de bolo. Container é o bolo pronto. Você pode fazer 10 bolos com a mesma receita.

7.3 Dockerfile: a receita da nossa imagem

É um arquivo texto que descreve **como construir** a imagem passo a passo. O nosso resumido:


```
# Estágio 1: BUILD
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/FormaturasFlow.Api/FormaturasFlow.Api.csproj src/FormaturasFlow.Api/
RUN dotnet restore src/FormaturasFlow.Api/FormaturasFlow.Api.csproj
COPY src/ src/
RUN dotnet publish -c Release -o /app/publish

# Estágio 2: RUNTIME (bem menor)
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
USER app
EXPOSE 8080
ENTRYPOINT ["dotnet", "FormaturasFlow.Api.dll"]
```

Por que dois estágios?

- O estágio "build" tem o SDK do .NET (~800MB) — precisa pra compilar.
- O estágio "runtime" tem só o runtime (~200MB) — o suficiente pra rodar.
- A imagem final é a **runtime**, sem o peso do SDK. Ela puxa o binário compilado do estágio de build.

7.4 Docker Compose: vários containers

Um app real tem múltiplas peças: API, banco, cache, worker. O **Compose** descreve tudo isso em um arquivo `docker-compose.yml`.

```
services:
  postgres:
    image: postgres:16-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks: [internal]

  api:
    image: ghcr.io/joaooof/formaturas-api:latest
    depends_on:
      postgres: { condition: service_healthy }
    ports:
      - "80:8080"
    networks: [edge, internal]

  watchtower:
    image: nickfedor/watchtower:latest
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

Um único `docker compose up -d` sobe tudo, na ordem certa, na rede certa.

Volumes: persistência

Containers são "efêmeros" — se você mata o container, o que tava dentro dele some. Pro banco isso é catástrofe. A solução é o **volume**: um pedaço do disco da máquina hospedeira que é montado dentro do container. Quando o container morre, o volume permanece.

Redes: isolamento

O nosso Postgres está numa rede chamada `internal` que é `internal: true`. Isso significa que só quem está nessa rede consegue falar com ele. A API está em `internal` (fala com Postgres) e em `edge` (fala com a internet). O Postgres nunca fica exposto na internet — ninguém de fora chega nele.

Parte VIII

Colocando de verdade num servidor na internet

8.1 VPS: um computador na nuvem

Uma **VPS** (Virtual Private Server) é um computador virtualizado que fica ligado 24h por dia num data center. Você aluga por mês. A nossa é uma Hostinger com Ubuntu 24.04 LTS.

Diferença pra "hospedagem compartilhada" tradicional (tipo Locaweb): você tem **controle total**. Instala Linux, instala Docker, roda o que quiser.

8.2 SSH: acesso remoto criptografado

Você não vai até o data center digitar coisa no servidor. Você usa **SSH** (Secure Shell): um protocolo criptografado pra digitar comandos remotamente.

```
ssh root@191.101.78.252
```

Chave pública vs chave privada

Senha é ruim. Um dia alguém adivinha. A alternativa é **par de chaves**:

- **Chave privada**: fica só no seu computador. NUNCA compartilha.
- **Chave pública**: você bota no servidor.

Elas são matematicamente relacionadas. Só quem tem a privada consegue "provar" que é dono da pública. Ninguém consegue adivinhar a privada a partir da pública (leva mais que a idade do universo com toda energia do planeta).

BOA PRÁTICA

Use `ed25519` (algoritmo moderno) e nunca use RSA de menos de 4096 bits. Nossa chave é `ed25519` — mais curta, mais rápida, mais segura.

8.3 Firewall: fechar tudo, abrir só o essencial

Por padrão, um Linux "cru" tem várias portas abertas. Cada porta aberta é uma superfície de ataque. Regra de ouro: **feche tudo, abra só o que precisa**.

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp      # SSH pra nós entrarmos
ufw allow 80/tcp      # HTTP pra API responder
ufw enable
```

8.4 Fail2ban: banindo quem chuta senha

Mesmo com SSH por chave, tem gente chutando senha na porta 22 o tempo todo (é bot varrendo a internet). O **fail2ban** monitora tentativas falhas e bane o IP por X minutos depois de Y tentativas.

Ele já vem com uma configuração padrão razoável. Só precisa ligar: `systemctl enable --now fail2ban`.

8.5 Usuário deploy (nunca rodar como root)

O `root` pode tudo. Se um app rodando como root é comprometido, o atacante ganha o servidor inteiro. Boa prática: criar um usuário sem privilégios (`deploy`), rodar o Docker como ele.

Depois de tudo configurado, ainda desabilitamos:

```
PermitRootLogin no          # root não pode logar via SSH
PasswordAuthentication no   # ninguém loga por senha, só chave
```


Parte IX

CI/CD: automação total do deploy

9.1 O que é CI/CD

- **CI (Continuous Integration)**: cada mudança é testada automaticamente. Se quebrar, você é avisado.
- **CD (Continuous Deployment)**: se passar nos testes, é publicada automaticamente.

Antes do CI/CD, o dev empacotava manualmente, entrava no servidor por FTP, subia arquivo. Deixava o servidor 1 hora offline. Errava. Voltava atrás.

Agora: `git push`. Em 2 minutos está no ar. Sem downtime perceptível.

9.2 GitHub Actions

É a plataforma de CI/CD do GitHub. Você escreve um arquivo YAML em `.github/workflows/` descrevendo os passos.

```
name: build-and-push
on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: ghcr.io/joaoof/formaturas-api:latest
```

Traduzindo: "quando alguém der push na main, num Ubuntu limpo, checka o código, builda a imagem Docker, publica no GHCR".

9.3 GHCR: onde a imagem mora

GitHub Container Registry. É como um Google Drive, mas pra imagens Docker. Você publica lá com um token; puxa de lá com o mesmo token. Free tier tem 500MB, que é bastante pra APIs pequenas.

9.4 Watchtower: o gancho da automação

Aqui tá a mágica. O Watchtower é um container que:

1. A cada 60s, vai no GHCR
2. Vê se a imagem que ele monitora tem uma "digest" nova
3. Se tiver: puxa, para o container antigo, sobe o novo

Se você combinar com o GitHub Actions, o fluxo completo é:

1. `git push`
2. GitHub Actions builda (60-90s)
3. Publica no GHCR
4. Watchtower detecta em $\leq 60s$
5. Auto-update

Total: ~2 minutos, sem SSH manual.

DOR REAL QUE VIVI

A imagem original [containrrr/watchtower](#) está **abandonada**. Ela usa uma API do Docker antiga que já não existe mais no Docker 29. Usamos o fork [nickfedor/watchtower](#) que é ativamente mantido. Se você não sabe disso, passa horas debugando "container restarting" sem saber por quê. Boa prática: **verifica se as ferramentas que você usa estão vivas**.

Parte X

Testes: como dormir tranquilo à noite

10.1 Por que testar?

Sem testes, cada mudança no código é uma aposta. Você *acha* que não quebrou nada, mas não sabe. Aí em produção seu cliente reclama porque um botão parou de funcionar.

Com testes, você **prova** que os fluxos essenciais funcionam. E a cada mudança, o teste roda de novo. Se quebrou algo, você descobre em segundos, não em produção.

10.2 Três níveis de teste

Unit test (unitário)

Testa uma unidade isolada: uma função, um método. Não fala com banco, não fala com rede. Ultra rápido (milissegundos). Ex: testar que o rateio de parcelas divide direito.

```
[Fact]
public void Divisao_Nao_Exata_Sobra_Centavos_Na_Ultima()
{
    var (parcela, resto) = Ratear(total: 100m, entrada: 0m, n: 3);
    parcela.Should().Be(33.33m);
    resto.Should().Be(0.01m);
}
```

Integration test (integração)

Testa peças juntas: API + banco de dados. Mais lento (segundos), mas prova que o sistema real funciona. A gente usa **Testcontainers**: sobe um Postgres real em Docker só pra rodar o teste.

```
[Fact]
public async Task Primeiro_Register_Vira_Super_Admin()
{
    var http = factory.CreateClient();
    var tok = await http.RegisterAsync("admin@ex.com", "SenhaForte1!", "Admin");
    tok.Roles.Should().Contain("super_admin");
}
```

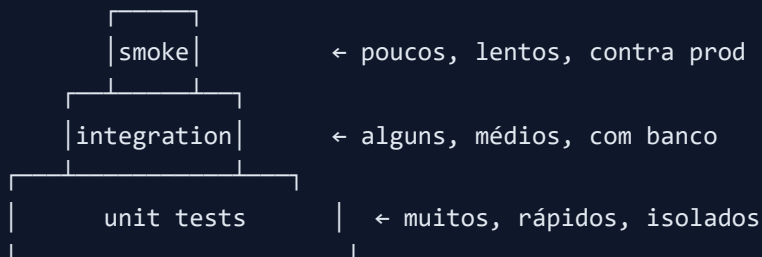
Smoke test (fumaça)

Testa contra o ambiente real (produção). Vale como "não morreu". Um script `bash` curto que faz:

```
curl http://api.formaturasflow.com/health
# espera Healthy
```

10.3 Pirâmide de testes

Muitos unitários. Alguns de integração. Poucos smokes. Assim:



Parte XI

Documentação viva: OpenAPI e Scalar

Documentar API em Word é o começo da morte. Em 2 semanas o Word está desatualizado.

OpenAPI (antigo Swagger) é uma *especificação*: um JSON que descreve todos os endpoints, parâmetros, respostas. O ASP.NET Core gera esse JSON automaticamente a partir do próprio código. Você mexe no código → doc atualiza sozinha.

Scalar é uma UI moderna que consome esse JSON e renderiza uma página web com todos os endpoints, com botão "Try it" pra você testar direto do navegador.

```
app.MapOpenApi();  
app.MapScalarApiReference();
```

Duas linhas. Documentação clicável em <http://seu-ip/scalar/v1>.

Parte XII

Boas práticas: o que ainda vamos fazer

12.1 HTTPS (crítico)

Hoje a API está em HTTP puro. Toda senha que passa vai em texto na rede. Compra domínio, aponta pra VPS, coloca um **Caddy** ou **Traefik** na frente que resolve certificado Let's Encrypt automaticamente.

12.2 Backup automático

Se o disco da VPS morrer amanhã, perdemos tudo. Cron diário:

```
docker exec postgres pg_dump -U user db | gzip > /backups/$(date +%F).sql.gz
```

E copia pro Backblaze B2 ou S3 (~R\$2/mês).

12.3 Rate limiting

`/auth/register` aberto = bot cria 10 mil contas em 30s. Rate limiting bloqueia N tentativas por IP por minuto.

```
builder.Services.AddRateLimiter(o => o.AddFixedWindowLimiter("auth", opt => {  
    opt.PermitLimit = 5;  
    opt.Window = TimeSpan.FromMinutes(1);  
}));
```

12.4 Refresh tokens

Access token de 60 min. Se vazar, atacante tem 60 min de acesso. Refresh token de 7 dias, guardado com hash no banco, permite **revogar**. Se suspeitar de vazamento, remove o refresh token → conta bloqueada.

12.5 Logging estruturado

Log de texto puro é ruim de buscar. Log estruturado (JSON) permite buscar por qualquer campo:

```
{ "timestamp": "...", "level": "Error", "traceId": "abc", "message": "...", "userId": "xyz" }
```

Combinado com **Serilog + Grafana Loki** (grátis), você tem busca poderosa em produção.

12.6 Observabilidade

Metrics + Traces + Logs. Com **OpenTelemetry** você exporta tudo pro Grafana Cloud (free tier generoso). Aí você vê "endpoint X ficou 2× mais lento na última hora" antes do cliente reclamar.

12.7 Feature flags

Uma flag no banco liga/desliga funcionalidade sem deploy. Ideal pra rollout gradual. Ex: "novo cálculo de rateio ativa pra 10% dos contratos novos".

12.8 Rotação de segredos

Se você acha que um segredo vazou (ou vazou mesmo — nosso PAT ficou nesse chat), **rotaciona**. Cria segredo novo, atualiza `.env`, revoga o velho. Faz isso periodicamente mesmo sem suspeita.

Parte XIII

Como pensar como um dev sênior

13.1 Decompor problemas

Um problema grande é intimidante. Divide em pedaços pequenos. "Fazer sistema de formaturas" é intimidante. "Fazer o endpoint `POST /alunos`" é factível em 30 minutos. Vai peça por peça.

13.2 Ler o erro inteiro

90% dos bugs se resolvem lendo a mensagem de erro com atenção. A stack trace te diz o arquivo e a linha exata. Sério. **Leia antes de perguntar.**

13.3 Chato é bom

Use tecnologias chatas, bem testadas, com comunidade grande. PostgreSQL em vez de "esse banco novo super rápido". C# em vez de "essa linguagem exótica". Não é sexy, mas você vai encontrar 10 mil respostas no Stack Overflow. Novidade em produção é dor.

13.4 Faça funcionar → faça direito → faça rápido

Nessa ordem. Primeiro faz o código funcionar minimamente. Depois refatora pra ficar limpo. Só otimiza performance quando tiver medindo — *premature optimization is the root of all evil*.

13.5 Aprenda por curiosidade, não por medo

Não decore listas de "10 padrões que todo dev precisa saber". Quando você encontrar um problema, aprenda o padrão que resolve ele. Fica na memória.

13.6 Programe todo dia, mesmo 30 min

Consistência bate intensidade. 30 min todo dia por 1 ano é infinitamente melhor que 8 horas num sábado por mês.

13.7 Escreva o que aprendeu

Blog, README no seu GitHub, notas no Notion. Escrever força você a organizar o pensamento. E dali a 2 anos você lê e agradece.

GRANDE RESUMO

Programação é **resolver problemas com código**. Sintaxe é secundária. Consegue explicar o problema em português claro? Metade do trabalho tá feito. Erro te ensina, chato te salva, consistência te leva longe.

Glossário

API

Application Programming Interface. Um "cardápio" que um sistema oferece para outros sistemas usarem.

Backend

A parte do sistema que roda no servidor. Não tem interface visual. Nossa API é o backend.

Bootstrap

O primeiro passo de configuração de um sistema, tipicamente automatizado por um script.

CI/CD

Integração Contínua / Entrega Contínua. Automação de testes e deploy a cada mudança.

Claim

Uma "declaração" dentro de um JWT. Ex: "esse token pertence ao user X".

Container

Um "pacote" isolado que contém uma aplicação e tudo que ela precisa pra rodar.

DI (Dependency Injection)

Padrão onde as dependências são "injetadas" pelo framework em vez de instanciadas manualmente.

DTO (Data Transfer Object)

Objeto simples pra transportar dados entre camadas. Não tem lógica, só campos.

Endpoint

Uma URL específica de uma API. Ex: `POST /api/v1/alunos`.

Entity Framework

ORM oficial da Microsoft pra .NET. Traduz classes C# em SQL.

Firewall

"Parede" que filtra tráfego de rede. UFW é o firewall que a gente usa.

Frontend

A parte visual. O site, o app. Consome o backend via API.

GHCR

GitHub Container Registry. Onde a gente publica as imagens Docker.

Hash

Função matemática de mão única. Usada pra guardar senhas, checksums, etc.

HMAC

Hash com uma chave secreta. Usado pra assinar JWTs.

HTTP

HyperText Transfer Protocol. O idioma da web moderna.

HTTPS

HTTP + criptografia. Todo tráfego é encriptado. Obrigatório em produção.

Idempotência

Propriedade de uma operação: fazer 1x ou 100x produz o mesmo resultado.

JSON

Formato de dados em texto. Chave-valor, aninhado. Universal na web.

JWT

JSON Web Token. Token assinado que carrega informações do usuário.

Migration

Um script versionado que altera a estrutura do banco de dados.

mTLS

Mutual TLS. Ambos os lados apresentam certificado. Extra segurança.

OpenAPI

Especificação padrão pra descrever APIs REST. Gera documentação automática.

ORM

Object-Relational Mapper. Traduz classes de programação em SQL.

PSP

Payment Service Provider. Empresa que processa pagamentos (Efí, Stripe, PagSeguro).

REST

Padrão de design de API baseado em recursos e verbos HTTP.

Role

Papel do usuário no sistema. Define o que ele pode fazer.

Scalar

UI moderna pra explorar OpenAPI. Substituto do Swagger UI.

SSH

Secure Shell. Protocolo pra acessar servidor remoto de forma segura.

Testcontainers

Biblioteca que sobe containers Docker durante os testes automatizados.

VPS

Virtual Private Server. Um servidor virtual alugado num data center.

Watchtower

Programa que atualiza containers automaticamente quando há nova imagem.

Webhook

URL sua que um serviço externo chama quando algo acontece.

Workflow

Um pipeline de CI/CD. No GitHub Actions, um arquivo YAML em `.github/workflows/`.

Recursos: onde continuar aprendendo

Cursos em português

- **Curso em Vídeo** (YouTube — Gustavo Guanabara): fundamentos de lógica, HTML/CSS, Python. Gratuito.
- **Rocketseat**: cursos gratuitos e pagos de Node.js, React, React Native, arquitetura.

- **Full Cycle** (fullcycle.com.br): Docker, Kubernetes, arquitetura distribuída. Nível intermediário/avançado.
- **Balta.io** (André Baltieri): C# e .NET com muita clareza.

Canais YouTube

- **Filipe Deschamps**: explicações claras sobre conceitos, boas práticas, mercado.
- **Fabio Akita**: perspectiva histórica e crítica sobre tecnologia.
- **Rocketseat**: séries práticas construindo coisas do zero.
- **Diolinux**: Linux e ferramentas de dev.
- **Ben Awad** (inglês): fullstack moderno com humor.

Livros que valem a leitura

- *The Pragmatic Programmer* — Dave Thomas & Andy Hunt. Bíblia da mentalidade.
- *Clean Code* — Robert C. Martin. Como escrever código legível (com pitadas de exagero, mas 80% ouro).
- *The Linux Command Line* — William Shotts. Grátis em linuxcommand.org.
- *Designing Data-Intensive Applications* — Martin Kleppmann. Depois de 1 ano codando, essa é a próxima parada.
- *Você não é uma máquina* — Isabela Delgado. Sobre carreira, saúde mental, burnout. Sério, leia.

Sites que você vai amar

- **Stack Overflow**: pra dúvidas técnicas. Copie o erro exato, cole no Google.
- **DevDocs.io**: documentação de várias linguagens agregada.
- **MDN Web Docs**: bíblia de HTML/CSS/JavaScript da Mozilla.
- **Microsoft Learn**: documentação de C# e .NET, muitos tutoriais.
- **Exercism.io**: exercícios de código com mentores humanos.

Comunidades brasileiras

- **Comunidade .NET Brasil** (Discord/Twitter)
- **Rocketseat Discord**: gigante, ativa, para JS/TS/Node/React.
- **DEV.to**: blog de devs, tem muita gente brasileira publicando.

O que fazer agora

1. Escolhe UM projeto pequeno e faz do zero (calculadora, todo list, API de recados).
2. Publica no GitHub, mesmo que feio. Consistência bate perfeição.
3. Escreve um README explicando o projeto. Isso força você a estruturar o que fez.
4. Faz o próximo. Um pouco mais ambicioso. Sempre um pouquinho maior.
5. Não compare seu capítulo 1 com o capítulo 20 dos outros.

ÚLTIMA COISA

Você já fez algo que 99% das pessoas do mundo nunca fizeram: **tem um sistema real, no ar, com CI/CD, testes, banco de dados, autenticação, integração com pagamento**. Isso é muito. Mesmo que ainda pareça mágica no seu ponto de vista, é seu.

Não desanima quando ver dev "sênior" fazendo coisas complexas. Ninguém nasce sabendo. A diferença é que ele começou 5 anos atrás e nunca parou. Você começou agora. Daqui a 5 anos, alguém vai olhar pra você e falar "cara, isso é mágica" — e vai ser porque você não parou.

Guia completo escrito com carinho para acompanhar o projeto **FormaturasFlow.Api**.

Fonte: [Poppins](#) · Código: [JetBrains Mono](#).

Para exportar como PDF: abra no navegador e aperte **Ctrl+P** → **Salvar como PDF**.